

Fine-tuning de Modelos

Módulo 4 · Deep Learning

SFT, PEFT, LoRA, QLoRA, instruction tuning y la decision prompting vs RAG vs fine-tuning para sistemas LLM en produccion.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Fine-tuning de Modelos de Deep Learning

El fine-tuning es el proceso de adaptar un modelo preentrenado a un dominio o tarea específica ajustando sus pesos con datos del dominio objetivo. Para las CNNs era el estándar desde 2016. Para los LLMs, el fine-tuning se ha convertido en la palanca más accesible para personalizar el comportamiento de modelos masivos sin los costes prohibitivos del preentrenamiento. Con PEFT, LoRA y QLoRA, es posible adaptar un modelo de 70B parámetros a un dominio específico con un presupuesto de hardware que cabe en un servidor estándar.

Este tema cubre tanto el fine-tuning supervisado clásico (adaptar el modelo a un dataset etiquetado de dominio) como las técnicas de fine-tuning eficiente (PEFT, LoRA, QLoRA) que hacen posible trabajar con modelos grandes en hardware limitado, y el fine-tuning por instrucciones (instruction tuning) que convierte modelos base en asistentes útiles.

Los tipos de fine-tuning y cuándo aplicarlos

Fine-tuning supervisado (SFT): aprender de ejemplos del dominio

El fine-tuning supervisado entrena el modelo sobre un conjunto de pares (input, output) del dominio objetivo. Para LLMs, esto típicamente toma la forma de pares (instrucción, respuesta): "clasifica este email como urgente o no urgente → urgente", "resume este contrato en 3 puntos → [resumen]". Con 1.000-10.000 ejemplos de alta calidad, el SFT puede especializar un modelo en un dominio específico de forma muy efectiva, especialmente si los datos de fine-tuning representan bien los tipos de consultas que el sistema recibirá en producción.

PEFT: adaptar sin entrenar todo el modelo

Parameter-Efficient Fine-Tuning (PEFT) es una familia de técnicas que adaptan el modelo entrenando solo una pequeña fracción de sus parámetros, manteniendo los pesos originales congelados. Las ventajas son enormes: menor coste de memoria (no hay que almacenar gradientes para todos los parámetros), menor riesgo de sobreajuste (menos parámetros = menos capacidad de memorizar el dataset), y la posibilidad de mantener múltiples adaptaciones del mismo modelo base cargando solo los pesos de la adaptación relevante.

LoRA: adaptación de bajo rango

LoRA (Low-Rank Adaptation, Hu et al., 2021) es la técnica PEFT más adoptada. La idea es elegante: en lugar de actualizar la matriz de pesos completa W ($d \times k$ parámetros), introduce dos matrices pequeñas A ($d \times r$) y B ($r \times k$) donde $r \ll d$. La actualización de W es $\Delta W = A \times B$. Con $r=16$ y $d=4096$, se pasa de 16.8M de parámetros a actualizar a 131K, una reducción de 128x. Durante la inferencia, los pesos LoRA pueden fusionarse con el modelo base ($W_{\text{nuevo}} = W + A \times B$) sin añadir latencia.

QLoRA (Quantized LoRA, Dettmers et al., 2023) combina LoRA con cuantización del modelo base a NF4 (4-bit normal float), reduciendo aún más el uso de memoria. Con QLoRA es posible hacer fine-tuning de Llama 3 70B en una sola GPU A100 de 80GB, algo que sería imposible con

fine-tuning completo (que requeriría ~420GB de VRAM para los pesos + gradientes + estados del optimizador).

SECCIÓN 3 · CÓMO FUNCIONA

El pipeline de fine-tuning con LoRA y QLoRA

El proceso estándar de fine-tuning de un LLM con QLoRA usando el framework Hugging Face + PEFT + bitsandbytes: cargar el modelo base en cuantización NF4 (BitsAndBytesConfig con `load_in_4bit=True`), aplicar `prepare_model_for_kbit_training` para preprocesar el modelo, configurar LoRA (LoraConfig con `r=16`, `lora_alpha=32`, `target_modules=["q_proj","v_proj"]`, `lora_dropout=0.05`), obtener el modelo PEFT (`get_peft_model`), y entrenar con el Trainer de Hugging Face o TRL (Transformer Reinforcement Learning).

Los módulos objetivo de LoRA más frecuentes en LLMs son las matrices de proyección de atención (`q_proj`, `k_proj`, `v_proj`, `o_proj`) y opcionalmente las matrices del FFN (`gate_proj`, `up_proj`, `down_proj`). Aplicar LoRA solo a las matrices de atención es el default conservador. Aplicarlo también al FFN aumenta la capacidad de adaptación a cambio de más parámetros a entrenar.

Instruction tuning: enseñar al modelo a seguir instrucciones

El instruction tuning (o SFT con datos de instrucción) es el proceso por el que un modelo base (entrenado para predecir el siguiente token) se convierte en un asistente que sigue instrucciones. El dataset de fine-tuning consiste en pares (instrucción, respuesta ideal) cubriendo una amplia variedad de tareas. Alpaca, Dolly, y Open Assistant son datasets públicos de instruction tuning. Para casos de uso específicos, construir un dataset de 500-2.000 pares instrucción-respuesta de alta calidad específicos del dominio suele ser suficiente para una especialización efectiva.

SECCIÓN 4 · EJEMPLOS REALES

- Fine-tuning de Llama 3 8B para extracción de cláusulas contractuales (firma legal): 1.500 pares (contrato, extracción estructurada en JSON) generados por Claude sobre contratos reales de la empresa. Fine-tuning con QLoRA (`r=16`) en 4 horas en una A100. Resultado: el modelo especializado supera a Claude 3 Haiku en la tarea específica y cuesta 10x menos en inferencia por la menor latencia del modelo de 8B vs los modelos frontier.
- Especialización de Mistral 7B para coding en Python interno (empresa de software): 8.000 ejemplos de código Python interno con patrones, convenciones y APIs propietarias de la empresa. El modelo fine-tuneado genera código que sigue automáticamente los estándares internos sin necesidad de instrucciones explícitas, reduciendo el tiempo de revisión de PRs.
- Fine-tuning de BERT para NER en textos médicos (hospital): 5.000 historias clínicas anotadas con entidades (medicamento, dosis, diagnóstico, procedimiento). BERT fine-tuneado en este dominio alcanza F1 de 0.91 en el test de entidades médicas, superando al modelo médico genérico (BioBERT) que sin fine-tuning dominio-específico obtiene 0.79.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Pensar que más datos de fine-tuning siempre es mejor. La calidad de los datos de fine-tuning es más importante que la cantidad. 500 ejemplos de alta calidad y diversidad superan a 5.000 ejemplos generados de forma descuidada. El proceso de curación y validación de los datos de fine-tuning es tan importante como el proceso de entrenamiento.

Error 2: No evaluar el catastrophic forgetting. El fine-tuning puede degradar las capacidades generales del modelo en dominios no representados en los datos de fine-tuning. Evalúa las capacidades generales del modelo antes y después del fine-tuning con benchmarks como MMLU o HellaSwag para detectar degradación significativa.

Error 3: Usar rank de LoRA demasiado alto innecesariamente. $r=16$ es suficiente para la mayoría de las tareas de fine-tuning. Usar $r=128$ o $r=256$ aumenta los parámetros entrenables y el riesgo de sobreajuste sin producir mejoras significativas en la mayoría de los casos. Empieza con $r=8$ o $r=16$ y escala solo si el rendimiento es insuficiente.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El proceso de evaluación de si el fine-tuning es necesario: primero, evalúa el modelo base con prompting avanzado (few-shot, CoT) en tu tarea. Si el rendimiento es suficiente para el caso de uso, no hay que hacer fine-tuning. Si el rendimiento es insuficiente después de optimizar el prompt, considera RAG para problemas de conocimiento o fine-tuning para problemas de estilo, formato o comportamiento específico.

Cuando el fine-tuning es la decisión correcta, el framework estándar es TRL (Transformer Reinforcement Learning) de Hugging Face, que encapsula SFT, RLHF y DPO con una API de alto nivel. Para casos de uso de clasificación y NER con BERT y sus variantes, el Trainer estándar de Hugging Face con `AutoModelForSequenceClassification` o `AutoModelForTokenClassification` es suficiente.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M4-T5 (Transfer learning): el fine-tuning es la aplicación de transfer learning a LLMs.
- M6 (Ingeniería de prompts): la decisión entre prompting y fine-tuning es central en el diseño de sistemas LLM.
- M8 (LLMOps): el versionado de modelos fine-tuneados, el regression testing y el monitoreo de catastrophic forgetting son aspectos operativos del fine-tuning en producción.

SECCIÓN 8 · RESUMEN EJECUTIVO

Fine-tuning adapta modelos preentrenados a dominios o tareas específicas. PEFT y LoRA permiten hacer fine-tuning de modelos grandes entrenando solo ~0.1-1% de los parámetros con matrices de bajo rango ($r=16$ es el valor estándar). QLoRA combina LoRA con cuantización NF4 para reducir la VRAM necesaria 4-8x. El instruction tuning crea asistentes a partir de modelos base con pares instrucción-respuesta. La calidad de los datos es más importante que la cantidad: 500 ejemplos excelentes superan a 5.000 mediocres. Evalúa siempre catastrophic forgetting. La decisión prompting vs RAG vs fine-tuning depende del tipo de problema: comportamiento/estilo → fine-tuning, conocimiento actualizado/privado → RAG, instrucciones más precisas → prompting.